Submission 3

Opcode Extraction from Malware Executables Using Ghidra

Group members

Arshdeep Kaur Januhe 1408202

Shilpa Merlin Joy 1411455

1. Introduction

The goal of this project was to extract Operational Codes (OpCodes) from a collection of malware executables associated with known Advanced Persistent Threat (APT) groups. Opcode sequences represent the low-level machine instructions executed by a program and are commonly used in malware classification and behavioral analysis.

To accomplish this task, malware binaries collected in Submission 2 were analyzed using the reverse-engineering framework GHIDRA. The extracted instruction mnemonics were exported as .opcode files and organized according to the malware names used in Submission 2 to ensure consistent dataset mapping.

The final output consists of a dataset of opcode sequences that can be used for later machine-learning-based malware classification.

2. Tools and Environment

The opcode extraction pipeline was implemented using the following tools:

Tool	Purpose
Ghidra	Static reverse-engineering and disassembly
Ghidra Script (Python,Java)	Automated extraction of opcode mnemonics
Kali Linux	Malware analysis environment
APKTool / SMALI tools	Processing Android APK samples
Ghidra Version 12.0.3	Headless analysis and scripting environment

GHIDRA was selected because it provides reliable static binary analysis, disassembly capabilities, and scripting support through Python and Java, making it suitable for automated opcode extraction. Opcode sequences were extracted using a Python script executed within the GHIDRA CodeBrowser environment after static analysis of each malware sample.

In addition, batch opcode extraction was implemented using the **GHIDRA Headless Analyzer**. The shell script run_extract.sh controlled the batch processing workflow, while the Java postScript OpcodeExport.java exported opcode mnemonics from the disassembled instructions. The list of malware executables was provided through input_files.txt, and execution logs were stored in the logs directory to support traceability and reproducibility of the extraction process.

3. Opcode Extraction Pipeline

The opcode extraction pipeline was designed to be consistent, reproducible, and automated. The process consists of several stages described below.

3.1 Malware Sample Preparation

The malware dataset collected in Submission 2 contained multiple executable formats including:

- Windows Portable Executable files (.exe)
- Linux Executable and Linkable Format binaries (.elf)
- Android application packages (.apk)

Windows PE and ELF binaries were analyzed directly in GHIDRA. Android APK files required preprocessing because they contain Dalvik bytecode rather than native machine instructions.

Android APK Processing

Android APK files contain Dalvik bytecode rather than native machine instructions, so they cannot be directly analyzed using GHIDRA's native disassembly workflow. The APK sample G0097_BouncingGolf_Golf.apk was therefore decompiled using APKTool, which converts the classes.dex bytecode into SMALI assembly files. The APK was decompiled using:

apktool d G0097_BouncingGolf_Golf.apk -o apk_decompiled

The generated SMALI files were then processed using a Python script that iterates through all .smali files and extracts the instruction mnemonics as opcode sequences. These opcodes were written to an output file with the .opcode extension to maintain consistency with the opcode datasets generated from other malware samples.

Example extracted opcodes include:

move-object, invoke-virtual,const-string,iget,return-void,goto,if-eqz

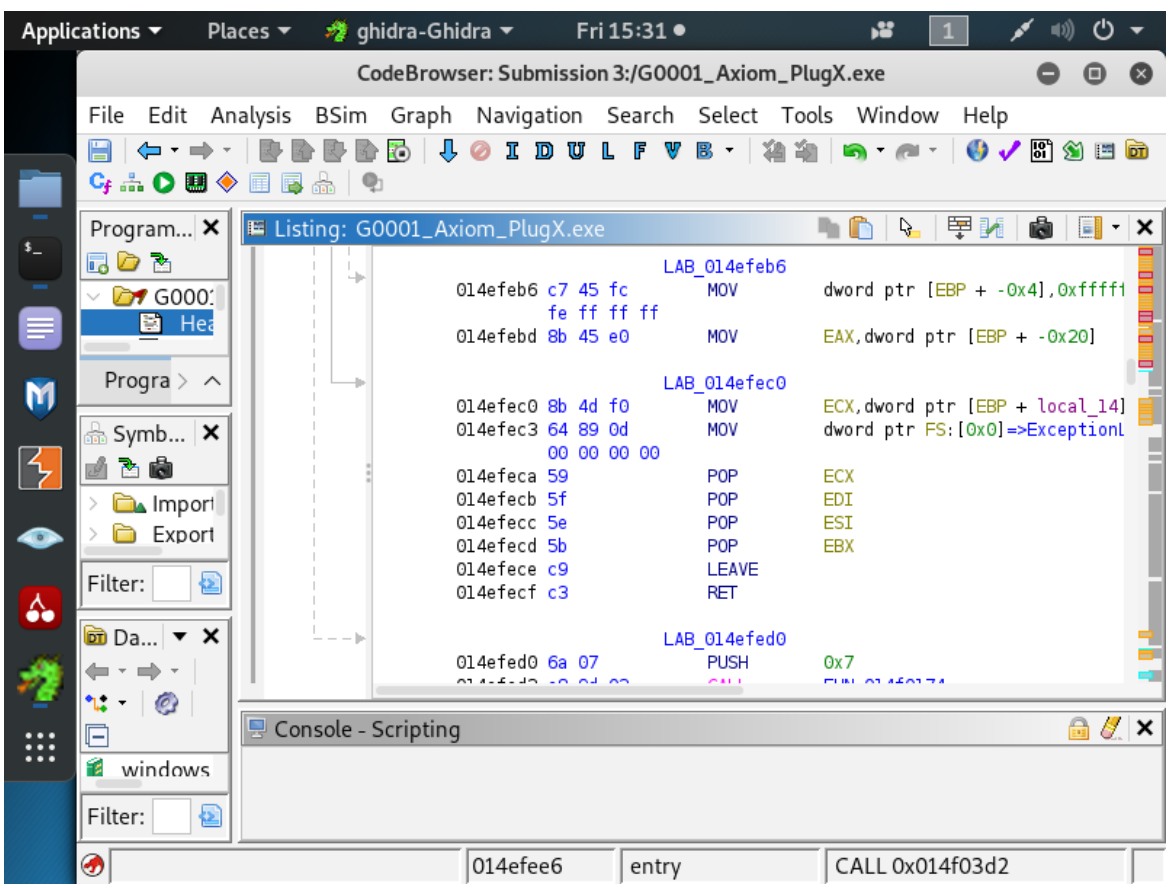
3.2 Static Disassembly Using GHIDRA

Each executable binary was imported into a GHIDRA project and analyzed using Auto-Analysis with the Standard Default analyzer configuration.

During this step, GHIDRA performs several static analysis tasks:

- Identification of executable sections
- Disassembly of machine code into assembly instructions
- Detection of function boundaries
- Identification of data references and control flow

This process converts raw machine code into readable assembly instructions containing opcode mnemonics, which form the basis of the extracted .opcode files. However, static disassembly has limitations. Malware may be packed, obfuscated, malformed, or only partially visible to static analysis, and GHIDRA may also report warnings such as unreadable bytes at certain addresses. These issues do not necessarily prevent opcode extraction, but they can reduce disassembly quality and limit the completeness of the visible instruction stream.



Above figure shows Ghidra CodeBrowser displaying disassembled instructions for G0001_Axiom_PlugX.exe after Auto-Analysis. This confirms that the executable was successfully imported and analyzed.

3.3 Automated Opcode Extraction

A custom Python script was implemented in GHIDRA to automatically extract opcode mnemonics from the disassembled instruction listing. The script iterates through all instructions in the program and retrieves the mnemonic of each instruction using:

```
instruction.getMnemonicString()
```

Each extracted opcode is written sequentially to an output file with the .opcode extension.

For each malware sample, the binary was imported into GHIDRA and analyzed using the Auto-Analysis feature with the default configuration. After disassembly was completed, the Python script was executed to extract opcode mnemonics from the program's instruction listing.
Example extracted opcode sequence: mov, push, call, cmp…

This process was repeated for each malware binary in the dataset. The resulting .opcode files were named according to the malware identifiers used in Submission 2 to maintain consistent mapping between the original binaries and the extracted opcode datasets.

3.4 Alternative Automated Extraction Pipeline

In addition to the Python-based opcode extraction script executed within the GHIDRA GUI environment, an alternative automated pipeline was implemented using the **GHIDRA Headless Analyzer**. This approach enables batch processing of multiple malware binaries without manual interaction.

The pipeline consists of two components:

1. **Opcode extraction script (Java)** executed within the GHIDRA scripting environment.
   The script iterates through the disassembled instructions of a program and extracts opcode mnemonics using:

   ```
   getMnemonicString()
   ```

   Each mnemonic is written sequentially into an output .opcode file.

2. **Batch automation script (Bash)** that invokes the GHIDRA Headless Analyzer to process multiple malware samples listed in an input file. The script automatically imports each binary, runs analysis, executes the opcode extraction script, and stores the results in the output dataset directory.

This approach allows scalable extraction across large malware datasets while maintaining consistent output formatting.

The complete scripts used in the extraction pipeline are included in the submission archive for reproducibility.

4. Limitations of Static Malware Analysis

4.1 Disassembly Limitations

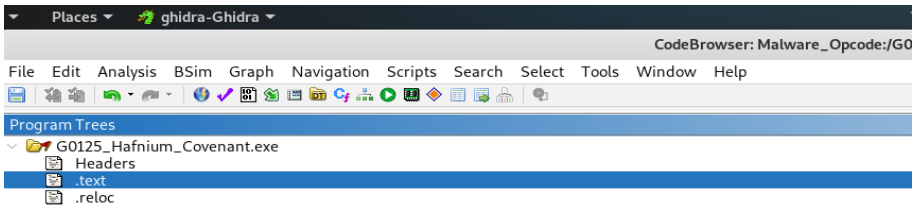
Opcode extraction in this project relies on static analysis, which examines the binary without executing it. Although static analysis is safe and efficient, it has several inherent limitations.

One limitation occurs when analyzing binaries that act as loaders or packed executables. For example, the malware sample **G0125_Hafnium_Covenant.exe** contains only minimal executable sections.

Example binary structure observed during analysis:

.text
.reloc

This minimal section layout may indicate that the binary acts as a loader that retrieves or decrypts the actual malicious payload during runtime. In such cases, static disassembly reveals only the loader instructions while the primary payload remains hidden until execution.



As a result, opcode extraction may capture only the visible instructions contained in the loader stub.

4.2 Dynamic Code Execution Indicators

Another limitation of static analysis is that it cannot observe instructions that are generated dynamically during program execution.

During analysis of the imported APIs within several malware samples, functions associated with runtime memory manipulation and process injection were observed. Examples include:

VirtualAlloc
WriteProcessMemory
CreateRemoteThread
OpenProcess
VirtualProtect

These APIs are commonly used by malware to perform **dynamic code execution**, where executable instructions are written into memory and executed during runtime.

Symbol Table [CodeBrowser: Malware_Opcode:/G0010_Turla_EPIC.exe]

File Edit Help

Symbol Table - (Filter settings matched 377 Symbols)

Name	Location	Type	Namespace	Source	Reference Count	Offcut Ref Count
HeapAlloc	External[0000bb48]	External Data	KERNEL32.DLL	Imported	11	0
HeapFree	External[0000bb54]	External Data	KERNEL32.DLL	Imported	19	0
GetFileSize	External[0000bb60]	External Data	KERNEL32.DLL	Imported	2	0
WaitForSingleObject	External[0000bb6e]	External Data	KERNEL32.DLL	Imported	3	0
VirtualFree	External[0000bb84]	External Data	KERNEL32.DLL	Imported	8	0
GetProcessHeap	External[0000bb92]	External Data	KERNEL32.DLL	Imported	2	0
CreateRemoteThread	External[0000bba4]	External Data	KERNEL32.DLL	Imported	2	0
OpenProcess	External[0000bbba]	External Data	KERNEL32.DLL	Imported	4	0
VirtualFreeEx	External[0000bbc8]	External Data	KERNEL32.DLL	Imported	3	0
Sleep	External[0000bbd8]	External Data	KERNEL32.DLL	Imported	7	0
ReadFile	External[0000bbe0]	External Data	KERNEL32.DLL	Imported	3	0
CreateFileW	External[0000bbec]	External Data	KERNEL32.DLL	Imported	2	0
GetProcAddress	External[0000bbfa]	External Data	KERNEL32.DLL	Imported	16	0
VirtualAlloc	External[0000bc0c]	External Data	KERNEL32.DLL	Imported	6	0
VirtualAllocEx	External[0000bc1c]	External Data	KERNEL32.DLL	Imported	4	0
GetExitCodeThread	External[0000bc2e]	External Data	KERNEL32.DLL	Imported	2	0
GetModuleHandleA	External[0000bc42]	External Data	KERNEL32.DLL	Imported	6	0
VirtualProtect	External[0000bc56]	External Data	KERNEL32.DLL	Imported	3	0
CloseHandle	External[0000bc68]	External Data	KERNEL32.DLL	Imported	16	0
WriteProcessMemory	External[0000bc76]	External Data	KERNEL32.DLL	Imported	4	0
GetCurrentProcess	External[0000bc8c]	External Data	KERNEL32.DLL	Imported	5	0
ConnectNamedPipe	External[0000bca0]	External Data	KERNEL32.DLL	Imported	2	0
CreateNamedPipeW	External[0000bcb4]	External Data	KERNEL32.DLL	Imported	2	0

Because these instructions are generated during execution rather than stored directly in the binary, they may not appear in the statically disassembled instruction listing used for opcode extraction.

Despite these limitations, static opcode extraction still captures the majority of instructions present in the executable and provides a useful representation of malware behavior.

5. Dataset Organization and Validation

All extracted opcode files were organized according to the naming convention used in Submission 2 to maintain a clear mapping between the original malware samples and the extracted opcode sequences. Each extracted file was saved using the original malware sample name with the additional .opcode extension, preserving a direct one-to-one relationship between the source executable and its corresponding opcode output file.

Input Executable	Output Opcode File
G0001_Axiom_PlugX.exe	G0001_Axiom_PlugX.exe.opcode
G0013_APT30_BACKSPACE.exe	G0013_APT30_BACKSPACE.exe.opcode
G0125_Hafnium_Covenant.exe	G0125_Hafnium_Covenant.exe.opcode

This naming convention was applied consistently across the dataset so that every .opcode file can be traced back to the original malware binary without ambiguity.

Verification checks were performed to ensure the correctness of the extraction process. Each processed malware executable was confirmed to produce a corresponding .opcode output file, and filenames were verified to follow the naming convention established in Submission 2. Selected binaries were manually reviewed in GHIDRA CodeBrowser to confirm successful disassembly before opcode export, and execution logs were examined to identify warnings, unreadable bytes, or unsupported formats.

The submission package is organized to clearly separate scripts, outputs, and documentation:

- Script/ – contains the automation and extraction scripts used in the opcode extraction process. This includes the batch automation script **run_extract.sh**, the GHIDRA headless extraction script **OpcodeExport.java**, the Python script used to extract opcode mnemonics from Windows and ELF binaries within the GHIDRA scripting environment(**extract_opcode.py**), and the Python script used to extract opcode sequences from Android SMALI files generated from APK decompilation(**extract_smali_opcode.py**)
- Raw_Extracted_Files/ – contains the generated .opcode files for each malware sample
- Submission3_Report.pdf

6. Conclusion

This project implemented a reproducible pipeline for extracting opcode sequences from malware binaries using GHIDRA. Through static disassembly and automated scripting, opcode datasets were generated for malware samples associated with multiple APT groups.

Although static analysis may not reveal instructions generated dynamically at runtime or hidden within packed payloads, the extracted opcode sequences provide a meaningful representation of the program's static instruction patterns.

The resulting opcode dataset will be used in the next stage of the project to perform n-gram feature extraction and machine learning classification of malware families.